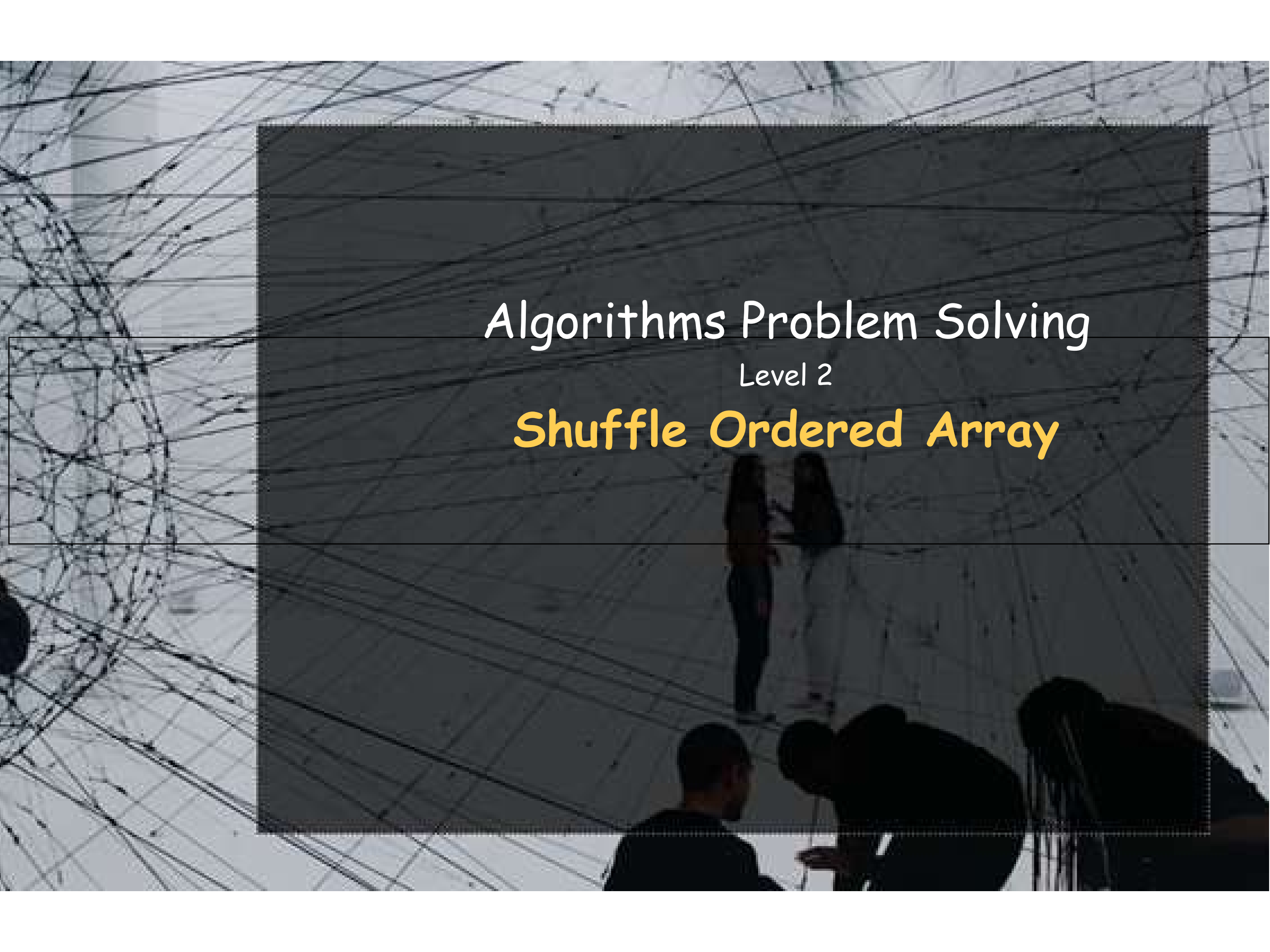


Algorithms Problem Solving

Level 2

Shuffle Ordered Array

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower portion of the image, the silhouettes of several people are visible, some standing and others sitting, suggesting a collaborative workspace or a public area. The overall color palette is muted, with greys, blues, and the warm tones of the light coming through the glass.

Algorithms Problem Solving

Level 2

Shuffle Ordered Array

Definition

- ◆ This program fills an array with ordered numbers from `1` to `N`.
- ◆ It prints the array before shuffling.
- ◆ It then randomly changes the positions of the elements.
- ◆ Finally, it prints the array after shuffling.

Explanation

- ◆ ``FillArrayWithNumbersFrom1ToN()`` fills the array with numbers starting from ``1`` up to ``N``.
- ◆ ``Swap()`` exchanges the values of two variables.
- ◆ ``ShuffleArray()`` changes the positions of the array elements randomly.
- ◆ For each element, a random index ``j`` is generated between ``0`` and ``i``.
- ◆ Then the current element ``arr[i]`` is swapped with ``arr[j]``.
- ◆ The important statement is:
$$\text{arr}[i] \leftrightarrow \text{arr}[j]$$
- ◆ This process continues until all elements have been processed.

Steps in main()

1. Ask the user for the number of elements.
2. Fill the array with numbers from `1` to `N`.
3. Print the array before shuffling.
4. Loop through the array.
5. Generate a random index between `0` and `i`.
6. Swap `arr[i]` with `arr[j]`.
7. Print the array after shuffling.

💡 Example

◆ Number of elements: `10`

◆ Before shuffle:

1 2 3 4 5 6 7 8 9 10

◆ During the shuffle, elements exchange positions randomly.

◆ One possible result:

2 4 6 3 10 1 7 8 5 9

◆ Another program run can produce a different order because the positions are randomized.

➡ Result

Array Elements before shuffle: 1 2 3 4 5 6 7 8 9 10

Array Elements after shuffle: 2 4 6 3 10 1 7 8 5 9

📌 Note

- ◆ `FillArrayWithNumbersFrom1ToN()` uses:

```
arr[i] = i + 1;
```

- ◆ Since the array index starts at `0`, adding `1` produces the values `1, 2, 3, ..., N`.

- ◆ `Swap()` uses a temporary variable to exchange two values:

```
int temp;
```

```
temp = A;
```

```
A = B;
```

```
B = temp;
```

- ◆ In `ShuffleArray()`:

```
int j = RandomNumber(0, i);
```

```
Swap(arr[i], arr[j]);
```

- ◆ `j` can be equal to `i`, which means the element can sometimes be swapped with itself.

- ◆ The array keeps exactly the same numbers after shuffling; only their positions change.

⊘ Warning

- ◆ ``arrayLength`` should not exceed the array size of ``100``.
- ◆ ``ReadArrayElements()`` is defined but not used.
- ◆ ``cmath`` and ``corecrt_math_defines.h`` are unnecessary for this program.
- ◆ ``rand()`` is pseudo-random and is not suitable for security-sensitive applications.
- ◆ The function name/comment ``swp`` is a spelling mistake; it should be ``swap``.



Comparison

▣ `FillArrayWithNumbersFrom1ToN()`

- ✓ Creates an ordered array.
- ✓ Stores numbers from `1` to `N`.
- ✓ Example: `1 2 3 4 5`.

▣ `ShuffleArray()`

- ✓ Changes the positions of the elements.
- ✓ Uses random indexes and `Swap()`.
- ✓ Example: `1 2 3 4 5` → `3 1 5 2 4`.

✦ Summary

- ✓ Fill the array with ordered numbers from `1` to `N`.
- ✓ Use `RandomNumber()` to generate random indexes.
- ✓ Use `Swap()` to exchange elements.
- ✓ `ShuffleArray()` changes the order without changing the values.
- ✓ The array contains the same numbers before and after shuffling, but in a different order.

Thank you 😊